    - Completely focus on the refactor to stabilize the foundation enough to build new features on.
    - Virtualize the back-end services so user-facing features could be developed against them while the refactor continued underneath it.

Service virtualization

Service virtualization is the practice of recreating the behavior of a system synthetically for isolated development and testing. Basically, you can emulate a dependency of a feature you need to build with virtualization, build and test the feature on top of the virtualized service, and deploy your new feature on top of the real service. This technique is particularly handy when consumer-facing features that need to iterate quickly depend on large back-end systems that can't change as fast.

According to the data they could see the following:

- Before the refactor the development team was averaging 12 points/developer/sprint (pds).
- The refactor team was still averaging 12 pds while changing a lot more code.
- The new-feature team was down to 4 pds.

These results led them to the following conclusions:

- Given their productivity was cut to one-third, if they could stop working for two sprints and get back to their average productivity on the third sprint, they would get the same amount of work done as if they worked at one-third productivity for three sprints.
- Based on their current velocity they could also infer that the refactor was slated to take another three sprints.

They decided that if they repurposed the entire new-feature team to tighten the team's automation for testing and virtualization for two sprints, they